

Django

עבודה על פרוייקט מסכם

TomerBu

נושאים להיום:

Users סיסמא מוצפנת, לא מוחזרת בבקשות **GET**

כפתורי התחברות/התנתקות

Permissions - הרשאות גלובליות ופרטניות

Authentication

Token Authentication

משתני סביבה

cors

הסתרה של הסיסמא בבקשות GET קבלה של הסיסמא מהלקוח בבקשות POST/PUT

```
class UserSerializer(ModelSerializer):  
    class Meta:  
        model = User  
  
        fields = ['id', 'username', 'email', 'password']  
  
        extra_kwargs = {'password': {'write_only': True}}
```

בעיה - אפשר למלא סיסמא 123456 לא עומד בדרישות הסף של הפרוייקט

```
{  
  "id": 5,  
  "username": "avi",  
  "email": "avi@gmail.com",  
  "password": "123456"  
}
```

פתרון - ביטוי רגולרי:

⋮ / `^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*]).{8,}$`

TEST STRING

abcDEF123!

בעיה - אפשר למלא סיסמא 123456 לא עומד בדרישות הסף של הפרוייקט

```
from rest_framework import serializers
from django.core.validators import RegexValidator

class UserSerializer(ModelSerializer):
    password = serializers.CharField(
        validators = [
            RegexValidator(
                regex=r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*]).{8,}$',
                message="Password ..."
            )
        ]
    )

    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password']
        extra_kwargs = {'password': {'write_only': True}}
```

הוראות שחזור הפרוייקט מגיטהב

<https://github.com/TomerBu/W270225MR/tree/main>

אחרי שמורידים מגיט - נריץ את הקוד הבא:

```
python -m venv env  
.\env\Scripts\activate
```

```
pip install -r requirements.txt
```

```
python manage.py runserver
```

לפני שמעלים לגיט - מקפיאים את הספריות:

```
pip freeze > requirements.txt
```

הסיסמא לא מוצפנת בבקשות POST

```
{  
  "id": 5,  
  "username": "avi",  
  "email": "avi@gmail.com",  
  "password": "123456"  
}
```

הסיסמא נשמרת כמחרוזת - נתקן

הסיסמא לא מוצפנת בבקשות POST

```
class UserSerializer(ModelSerializer):  
    password = serializers.CharField(  
        validators=[  
            RegexValidator(  
                regex=r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*]).{8,}$',  
                message="Password must be at least 8..."  
            )  
        ],  
        write_only=True  
    )
```

נדרוס את הפעולה create
נשתמש מתודה בשם create_user שמצפינה את הסיסמא

```
def create(self, validated_data):  
    user = User.objects.create_user(**validated_data)  
    return user
```

```
class Meta:  
    model = User  
    fields = ['id', 'username', 'email', 'password']  
    extra_kwargs = {'password': {'write_only': True}}
```


בעיה: הסיסמא לא מוצפנת בבקשות PUT

הסיסמא נשמרת כמחרוזת - נתקן

```
class UserSerializer(ModelSerializer):
    password = serializers.CharField(
        validators=[
            RegexValidator(
                regex=r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*]).{8,}$',
                message="Password must be at least 8 characters ..."
            )
        ],
        write_only=True
    )

    def create(self, validated_data):
        user = User.objects.create_user(**validated_data)
        return user

    def update(self, instance, validated_data):
        # remove the password from the dictionary
        password = validated_data.pop('password', None)

        # iterate over the dict and set the attributes of the user instance
        for key, value in validated_data.items():
            setattr(instance, key, value)

        instance.set_password(password)
        instance.save()
        return instance

class Meta:
    model = User
    fields = ['id', 'username', 'email', 'password']
    extra_kwargs = {'password': {'write_only': True}}
```

איזה פעולות מותר לבצע

Permissions:

Authentication

זיהוי של המשתמש

“המשתמש הוא תומר”

Authorization

איזה פעולות מותר לבצע

האם מותר ל“תומר” למחוק יוזרים אחרים???

הוספת כפתורי התחברות/התנתקות

api/urls.py

```
from api.views import TagViewSet, PostViewSet, CommentViewSet, PostUserLikesViewSet,
UserProfileViewSet, UserViewSet
from rest_framework.routers import DefaultRouter

from django.urls import path, include
router = DefaultRouter()
router.register(r'tags', TagViewSet, basename='tags')
router.register(r'posts', PostViewSet, basename='posts')
router.register(r'comments', CommentViewSet, basename='comments')
router.register(r'post-user-likes', PostUserLikesViewSet, basename='post-user-likes')
router.register(r'user-profiles', UserProfileViewSet, basename='user-profiles')
router.register(r'users', UserViewSet, basename='users')
urlpatterns = router.urls

# add login links to browsable api:
urlpatterns += [
    path('api-auth/', include('rest_framework.urls')),
]
```

הוספת כפתורי התחברות/התנתקות

Permissions

☒ Active

Designates whether this user s

☒ Staff status

Designates whether the user c

☒ Superuser status

Designates that this user has a

Groups:

קיימים במערכת באדמין פאנל

Permissions:

הגדרות כלליות לכל הAPI
בקובץ settings.py

```
REST_FRAMEWORK = {  
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.IsAuthenticated',  
    ],  
}
```

```
'rest_framework.permissions.AllowAny',
```

Permissions:

הגדרות פרטניות לview מסוים:

```
from rest_framework.permissions import AllowAny

class TagViewSet(ModelViewSet):
    queryset = Tag.objects.all()
    serializer_class = TagSerializer
    permission_classes = [AllowAny]
```

Permissions:

הגדרות פרטניות לview מסוים:

```
from rest_framework.permissions import IsAuthenticatedOrReadOnly

class TagViewSet(ModelViewSet):
    queryset = Tag.objects.all()
    serializer_class = TagSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]
```


Permissions:

הגדרת מחלקות משלנו לPermissions

```
from rest_framework import permissions

class IsAdmin(permissions.BasePermission):
    """
    Custom permission to only allow admin users to edit it.
    """

    def has_permission(self, request, view):
        return request.user and request.user.is_staff and request.user.is_superuser

class IsStaff(permissions.BasePermission):
    """
    Custom permission to only allow admin users to edit it.
    """

    def has_permission(self, request, view):
        return request.user and request.user.is_staff
```


Permissions:

הרשאות לפעולות על כל הקבוצה:

```
def has_permission(self, request, view):
```

הרשאות לפעולות עריכה ומחיקה על פריט בודד:

```
def has_object_permission(self, request, view, obj):
```

Permissions:

api/permissions.py

```
from rest_framework import permissions

class CommentOwnerOrReadOnly(permissions.BasePermission):
    def has_object_permission(self, request, view, obj):

        # req.user is the user object
        # obj is the comment object

        if request.method in permissions.SAFE_METHODS:
            return True

        if hasattr(obj, 'author') and hasattr(obj.author, 'user'):
            return obj.author.user == request.user

        return False
```

api/views.py

```
class CommentViewSet(ModelViewSet):
    queryset = Comment.objects.all()
    serializer_class = CommentSerializer
    permission_classes = [CommentOwnerOrReadOnly]
```

Permissions:

רק מנהל יכול לכתוב תגובות
ולקוחות יכולים לקרוא תגובות אך לא לערוך או למחוק

api/permissions.py

```
class PostsPermission(permissions.BasePermission):  
    def has_permission(self, request, view):  
        if request.method in permissions.SAFE_METHODS:  
            return True  
        return request.user and request.user.is_staff and request.user.is_superuser
```

```
class TagsPermission(PostsPermission):
```

```
    """
```

```
    Inherits from PostsPermission to apply the same rules for Ta
```

```
    """
```

אותן הרשאות כמו לפוסטים:
נבצע ירושה.

api/views.py

```
class PostViewSet(ModelViewSet):  
    queryset = Post.objects.all()  
    serializer_class = PostSerializer  
    permission_classes = [PostsPermission]
```

Permissions:

```
class CommentOwnerOrReadOnly(permissions.BasePermission):  
  
    # /api/comments  
    def has_permission(self, request, view):  
        if request.method in permissions.SAFE_METHODS:  
            return True  
        return request.user and request.user.is_authenticated  
  
    # /api/comments/{id}  
    def has_object_permission(self, request, view, obj):  
  
        if request.method in permissions.SAFE_METHODS:  
            return True  
  
        if hasattr(obj, 'author') and hasattr(obj.author, 'user'):  
            return obj.author.user == request.user  
  
        return False
```

Views:

```
from api.models import Tag, Post, PostUserLikes, User, UserProfile, Comment
from api.permissions import CommentOwnerOrReadOnly, IsAdmin, PostUserLikesPermission, PostsPermission, TagsPermission,
UserProfilePermission
from api.serializers import TagSerializer, CommentSerializer, PostUserLikesSerializer, PostSerializer, UserProfileSerializer,
UserSerializer
from rest_framework.viewsets import ModelViewSet
from rest_framework.permissions import IsAuthenticatedOrReadOnly

class TagViewSet(ModelViewSet):
    queryset = Tag.objects.all()
    serializer_class = TagSerializer
    permission_classes = [TagsPermission]

class PostViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    permission_classes = [PostsPermission]

class CommentViewSet(ModelViewSet):
    queryset = Comment.objects.all()
    serializer_class = CommentSerializer
    permission_classes = [CommentOwnerOrReadOnly]

class PostUserLikesViewSet(ModelViewSet):
    queryset = PostUserLikes.objects.all()
    serializer_class = PostUserLikesSerializer
    permission_classes = [PostUserLikesPermission]

class UserProfileViewSet(ModelViewSet):
    queryset = UserProfile.objects.all()
    serializer_class = UserProfileSerializer
    permission_classes = [UserProfilePermission]

class UserViewSet(ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [IsAdmin]
```

JWT:

<https://django-rest-framework-simplejwt.readthedocs.io/en/latest/#simple-jwt>

Installation

Simple JWT can be installed with pip:

```
pip install djangorestframework-simplejwt
```

```
pip freeze > requirements.txt
```

JWT:

<https://django-rest-framework-simplejwt.readthedocs.io/en/latest/#simple-jwt>

settings.py

```
REST_FRAMEWORK = {  
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',  
  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.IsAuthenticated',  
    ],  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework.authentication.SessionAuthentication',  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ],  
}
```

התחברות לממשק משתמש של API

עבור מערכות חיצוניות כגון רזקט

JWT:

<https://django-rest-framework-simplejwt.readthedocs.io/en/latest/#simple-jwt>

api/auth.py

```
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework_simplejwt.views import TokenObtainPairView

class MyTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)

        # Add custom claims
        token['isadmin'] = user.is_superuser
        return token
```

```
#jwt = MyTokenObtainPairSerializer.get_token(user)
```


מחלקת עזר להנפקת JWT

```
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework_simplejwt.views import TokenObtainPairView

class MyTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)

        # Add custom claims
        token['isadmin'] = user.is_superuser
        return token

def get_jwt(user):
    jwt = MyTokenObtainPairSerializer.get_token(user)
    return jwt
```

נקודות קצה - להרשמה והתחברות:

```
class AuthViewSet(ViewSet):  
    queryset = User.objects.all()  
    serializer_class = UserSerializer  
    permission_classes = [AllowAny]  
  
    @action(detail=False, methods=['post', 'get'])  
    def login(self, request):  
        # Implement login logic here  
        pass
```

נקודות קצה - להרשמה והתחברות:

```
from rest_framework.auth_token.serializers import AuthTokenSerializer
from api.auth import get_jwt
```

```
@action(detail=False, methods=['post', 'get'])
def login(self, request):
    serializer = AuthTokenSerializer(
        data=request.data, context={'request': request}
    )

    serializer.is_valid(raise_exception=True) # 401

    user = serializer.validated_data['user']
    jwt = get_jwt(user)
    return Response(jwt)
```

נקודות קצה - להרשמה והתחברות:

```
from api.serializers import UserSerializer
from api.auth import get_jwt
```

```
@action(detail=False, methods=['post', 'get'])
def register(self, request):
    serializer = UserSerializer(
        data=request.data
    )

    serializer.is_valid(raise_exception=True) # 401
    # saves the user and returns the user instance
    user = serializer.save()

    jwt = get_jwt(user)

    return Response(
        {
            "message": "User registered successfully",
            **jwt,
            "user": serializer.data
        }, status=status.HTTP_201_CREATED
    )
```

נקודות קצה - להרשמה והתחברות:

```
class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = AuthTokenSerializer
    permission_classes = [AllowAny]

    def list(self, request):
        return Response({
            "login": 'http://localhost:8000/api/auth/login/',
            "register": 'http://localhost:8000/api/auth/register/'
        })

    @action(detail=False, methods=['post', 'get'])
    def login(self, request):
        serializer = AuthTokenSerializer(
            data=request.data, context={'request': request}
        )

        serializer.is_valid(raise_exception=True) # 401

        user = serializer.validated_data['user']
        jwt = get_jwt(user)
        return Response(jwt)

    @action(detail=False, methods=['post', 'get'])
    def register(self, request):
        serializer = UserSerializer(
            data=request.data
        )

        serializer.is_valid(raise_exception=True) # 401
        # saves the user and returns the user instance
        user = serializer.save()

        jwt = get_jwt(user)

        return Response(
            {
                "message": "User registered successfully",
                **jwt,
                "user": serializer.data
            }, status=status.HTTP_201_CREATED
        )
```

נקודות קצה - להרשמה והתחברות:

```
class AuthViewSet(ViewSet):
    queryset = User.objects.all()
    serializer_class = AuthTokenSerializer
    permission_classes = [AllowAny]

    def list(self, request):
        return Response({
            "login": 'http://localhost:8000/api/auth/login/',
            "register": 'http://localhost:8000/api/auth/register/'
        })
```

urls.py

נתיב VIEW

```
from api.views import AuthViewSet, TagViewSet, PostViewSet, CommentViewSet,
PostUserLikesViewSet, UserProfileViewSet, UserViewSet
from rest_framework.routers import DefaultRouter

from django.urls import path, include
router = DefaultRouter()
router.register(r'auth', AuthViewSet, basename='auth')
router.register(r'tags', TagViewSet, basename='tags')
router.register(r'posts', PostViewSet, basename='posts')
router.register(r'comments', CommentViewSet, basename='comments')
router.register(r'post-user-likes', PostUserLikesViewSet, basename='post-user-likes')
router.register(r'user-profiles', UserProfileViewSet, basename='user-profiles')
router.register(r'users', UserViewSet, basename='users')
urlpatterns = router.urls

# add login links to browsable api:
urlpatterns += [
    path('api-auth/', include('rest_framework.urls')),
]
```


לוגין

JWT Decoder JWT Encoder

Paste a JWT below that you'd like to decode, validate, and verify.

ENCODED VALUE

☐ Enable auto-focus

JSON WEB TOKEN (JWT)

COPY

CLEAR

Valid JWT

Invalid Signature

[eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tldl90eXBldjoiYWNjZXNzIiwiaXhwIjoxNzUzMjQ0ZyxcCjpwYXQiOjE3NTcyNDQ0NjEsImp0aSI6ImJmYmYzZDJaInZMwMjQ4YmZiYTc3NTA4ZGRiYzE3NjBkIiwidXNlcld9pZCI6IjEiLCJpc2FkbWluIjp0cnVlfQ.qvGr_HPM6Cs6tmgSikcKhodJAjkbL_baZm8aaMMII-o](#)

DECODED HEADER

JSON

CLAIMS TABLE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

DECODED PAYLOAD

JSON

CLAIMS TABLE

```
{
  "token_type": "access",
  "exp": 1757244761,
  "iat": 1757244461,
  "jti": "bfbf3d2b730248bfbfa77508ddbc1760d",
  "user_id": "1",
  "isadmin": true
}
```


שימוש בJWT כדי לגשת למשאב:

[HTTP](#) Blog Project API / api / users / **users list**

GET

▼

`{{baseUrl}}/api/users/`

Params Authorization ● Headers (10) Body Scripts Settings

Headers 👁 8 hidden

	Key	Value	Description
⋮ ☒	Accept	application/json	
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1bI90eXBII...	
	Key	Value	Description

זמן חיים של JWT

```
from datetime import timedelta

SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(days=30)
}
```

לתיקון:

Text	Nice read
Post	Post: Database Migrations by admin
Author	Profile of admin
Reply to	-----

המחבר של post וcomment
צריך להלקח מהJWT

CORS headers

```
pip install django-cors-headers
```

```
pip freeze > requirements.txt
```

settings.py

```
INSTALLED_APPS = [  
    ...,  
    "corsheaders",  
    ...,  
]
```

settings.py

```
MIDDLEWARE = [  
    ...,  
    "corsheaders.middleware.CorsMiddleware",  
    "django.middleware.common.CommonMiddleware",  
    ...,  
]
```

CORS headers

settings.py

```
# public api:
# CORS_ALLOW_ALL_ORIGINS = True

CORS_ALLOWED_ORIGINS = [
    "https://example.com",
    "https://sub.example.com",
    "http://localhost:5173",
    "http://127.0.0.1:5173",
]
```

```
# public api:
CORS_ALLOW_ALL_ORIGINS = True
```

הרשימה של הכתובות כאן:

רשימה של לקוחות שמורשים להשתמש בapi

אם כתובת של צד לקוח לא כתובה כאן
CORS ERROR יקבל

אם רוצים API ציבורי:
לא מגבילים כתובות.